

TECHNICAL REPORT

Football Player Tracking System

Computer Graphics and Image Processing

Assignment

N.H.M.S.De Silva

ASP/2023/112

1. Introduction

This report is about a football player tracking system that I made using image processing techniques. The main idea is to take a short football video from YouTube, process the video frames, and then track a selected player across those frames. I used Python and OpenCV to build this system.

Football is a very fast sport. The players move quickly and the camera also move. So it is not easy to track one player in a video. This is why image processing is useful here. We can improve the image quality, find the player in each frame, and then follow that player from frame to frame.

The system I made has three main parts. First is image enhancement, where I improve the quality of the video frames. Second is segmentation, where I separate the player from the background. Third is tracking, where I follow the player across different frames.

2. Project Objectives

The main objectives of this project are:

- To download a short football match video from YouTube and extract individual frames from it.
- To improve the quality of the extracted frames using image enhancement techniques.
- To separate or isolate the football player from the background using segmentation methods.
- To track the selected player across multiple video frames using a tracking algorithm.
- To test the system on different match videos and analyse the results.

3. Tools and Technologies Used

For this project I used the following tools and libraries:

- Python – the main programming language I used to write all the code.
- OpenCV (cv2) – this is a computer vision library. It helped me to read videos, process frames, do segmentation and tracking.
- NumPy – this is a library for handling numbers and arrays. I used it to do calculations on image pixel values.
- Jupyter Notebook – I wrote and run all my code in Jupyter Notebook because it is easy to see the output step by step.
- GitHub – I uploaded all my code and output files to GitHub so they can be accessed easily.

4. Technical Task Description

4.1 Video Preparation

I downloaded a short football match video from YouTube. The video is only about 2 to 5 seconds long because that is enough to show player movement and to test the tracking system. To download the video I used an online YouTube video downloader tool. The video was saved in MP4 format on my computer.

Sport Selected – FootBall

Selected Youtube video link :

Video 01: <https://youtu.be/QXhV148EryQ?si=Fa4S2nJgoQyJwBzM>

Video 02: https://youtu.be/yqTZZkPU85o?si=OwVc3G4P_FwUb3xK

Video 03: <https://youtu.be/wRAUnwpRfgk?si=F2rcQbgfaNryef4I>

Test Dataset :

Match	Resolution	Frame Rate	Frames Tested	Duration Used
Match 01	1280x720	29.92 fps	560	17 s
Match 02	1280x720	29.91 fps	484	15 s
Match 03	1280x720	29.92 fps	496	16 s

Video Format Conversion

The video I downloaded was already in MP4 format. MP4 format is supported by OpenCV directly, so I did not need to convert it to another format. OpenCV can read MP4 files using the `cv2.VideoCapture()` function. So this step was not needed for my video.

Frame Extraction Process

After I got the video, I wrote a Python code in Jupyter Notebook to extract individual frames from the video. I used the OpenCV library for this. The basic process is:

- Open the video file using `cv2.VideoCapture()`.
- Read each frame one by one using `cap.read()` inside a loop.
- Save each frame as a .jpg image file in the frames folder.
- Stop when all frames are extracted or when the video ends.

The extracted frames are saved in the frames/ folder in my GitHub repository. Each frame is a single image from the video.

4.2 Image Enhancement Pipeline

Before I do segmentation and tracking, I need to improve the quality of the frames. Football videos sometimes have problems like motion blur (because players move fast), low contrast, or noise. So I designed an image enhancement pipeline to fix these problems.

Enhancement Pipeline Steps

Stage	Method	Purpose
1	Convert to Grayscale	Simplify the image for processing
2	Gaussian Blur	Reduce noise in the frame
3	CLAHE (Contrast Enhancement)	Improve brightness and contrast
4	Sharpening Filter	Make edges and player outline clearer
5	Save Enhanced Frame	Store the improved image

Stage 1 – Grayscale Conversion

First I convert each colour frame to grayscale using `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)`. Grayscale images are simpler to process because they have only one channel instead of three (Red, Green, Blue). This makes the later steps faster.

Enhancement Pipeline Flowchart

Input Frame



Resize



Peper noise



Histogram Equalization



Sharpening Filter



Enhanced Frame

4.3 Segmentation Pipeline

Segmentation means separating the player from the background. This is one of the hardest parts of the project because the football field background looks similar in many frames. I used background subtraction method for my segmentation.

Mathematical Background

Background subtraction works by comparing each frame with a background model. If a pixel is very different from the background model, it is considered as a foreground object (like a moving player). The formula is:

$$\text{Foreground}(x,y) = | \text{Frame}(x,y) - \text{Background}(x,y) | > \text{Threshold}$$

If the difference is bigger than the threshold value, the pixel is marked as foreground (white). If the difference is small, it is background (black). The result is a binary mask where white pixels show the player and black pixels show the background.

Segmentation Pipeline Flow

Enhanced Frame



HSV Color Conversion



Color Thresholding



Mask Inversion



Morphological Filtering



Swimmer Segmentation

Segmentation Challenges in Sports Videos

There are several challenges in sports video segmentation:

- The football field has mostly one colour (green), which make it easy for background subtraction to work. But sometime the player colour is also similar.
- Fast player movement can cause motion blur in the frame, making the contours unclear.
- When two players are close together, they can be detected as one object.

4.4 Player Tracking Implementation

After segmentation, I implemented a tracking algorithm to follow the selected player across all the frames. I used bounding box tracking method with OpenCV.

How the Tracking Works

The tracking process works in the following way:

- In the first frame, I manually define a bounding box (rectangle) around the player I want to track. This is the initial position.
- The tracker then learn the appearance of the player inside that bounding box.

- In each next frame, the tracker searches for the player and update the bounding box position.
- A green rectangle is drawn around the player in each frame to show the tracking result.
- I save each tracked frame as an image to show the tracking progression.

5. Experimental Testing

Test Dataset :

Match	Resolution	Frame Rate	Frames Tested	Duration Used
Match 01	1280x720	29.92 fps	560	17 s
Match 02	1280x720	29.91 fps	484	15 s
Match 03	1280x720	29.92 fps	496	16 s

5.2 Algorithm Testing

Enhancement Pipeline Testing

I applied the enhancement pipeline to all three match videos. I compared the original frames with the enhanced frames to see if the quality improved. In Match 01, the CLAHE made the player colours more visible. In Match 03, the sharpening helped to see the player outline even in the dark video.

Segmentation Testing

I tested the MOG2 segmentation on all three matches. In Match 01, the segmentation work very well because the background was simple (green grass). In Match 02, there was some extra noise because of the crowd in the background. In Match 03, camera movement cause some incorrect detection.

6. Results and Analysis

6.1 Qualitative Results

The qualitative results show the visual output of each stage of my pipeline.

Original vs Enhanced Frames

After applying my enhancement pipeline, the frames look much clearer than the original. The CLAHE improved the brightness in dark areas. The Gaussian blur removed some noise. The sharpening made the player outline more visible. These improvements help the segmentation step to work better.

Segmentation Results

The segmentation step successfully separated the player from the background in most frames. The foreground mask shows the player as white and the background as black. The contour detection then identifies the player shape and the bounding box is drawn around it.

In Match 01, the segmentation was very accurate. In Match 03, there was some extra noise in the mask but the player was still detected correctly.

7. Discussion and Interpretation

7.1 Enhancement Techniques

The image enhancement pipeline improves the frame quality before segmentation. CLAHE was the most useful step. In Match 01, it made the green field and player jersey more clear. However in Match 03, even with enhancement the low lighting and camera movement made it hard to get perfect results.

Gaussian blur was also helpful to reduce noise, but sometime it made the image too smooth and the player outline was less sharp. That is why I added the sharpening step after the blur.

8. Final Conclusion and Recommendations

8.1 Overall Effectiveness

The football player tracking system I made work successfully for most of the test cases. The three-stage pipeline (enhancement, segmentation, tracking) was effective in detecting and tracking a single player across short video clips.

8.2 Limitations

The system has some limitations:

- Fast motion cause motion blur in frames, which reduce segmentation accuracy.
- Camera movement confuse the background subtraction algorithm and cause false detections.
- When two players are very close to each other, they can be merged into one detected object.
- The system only track one player at a time. Multi-player tracking is not implemented.
- Lighting changes in Match 03 reduced the overall tracking performance.

8.3 Suggested Improvements

For future improvements, I suggest the following:

- Use deep learning methods like YOLO for better player detection. YOLO can detect multiple players at the same time and is more accurate.
- Use optical flow method to handle fast camera movement better.
- Implement multi-object tracking to track all players in the video, not just one.
- Use a higher quality video with higher frame rate for better results.
- Add a re-identification module so the tracker can find the player again after occlusion.

8.4 Real World Applicability

This system can be useful in real-world sports analytics. Football teams can use tracking systems to analyse player movement, speed, and position during a match. This information can help coaches to make better training plans and match strategies. Broadcasting companies can also use such systems to automatically highlight specific players in a video.

Even though my system is simple and basic, it shows that image processing techniques can be used for sports analytics. With more development, it can become a useful tool for real football analysis.

9. References

Git Hub Link: https://github.com/Maleeshaahh/Image_processing_assignment